CampusDesk — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and the four patterns

One file per job, and each stage is one LCEL pattern:

- `src/schemas.py` — `TicketCategory` (category as a `Literal`, reason, `sure: bool = True`) and `TicketRecord` (with `used_fallback`, `needs_human_review`, `review_reasons` defaulted).
- `src/chains/classifier.py` — `prompt | model.with_structured_output(TicketCategory)`.
- `src/chains/lookups.py` — `RunnableParallel` of three `RunnableLambda`s.
- `src/chains/replies.py` — `prompt | model | StrOutputParser()`.
- `src/chains/fallback.py` — the two `.with_fallbacks()` wrappers.
- `src/services/pipeline.py` — `handle_ticket`: classify → look up → reply (stream) → build the record; plus `review_reasons`.

Build order: classifier + routing → parallel lookups → retry + fallback → streaming + the record.

Hint 2 — structured classify, and parallel lookups

The classifier never parses free text — the model must answer in the shape:

```
structured = prompt | model.with_structured_output(TicketCategory)
```

`Literal["enrollment","financial_aid","it_support"]` means a wrong category is a `ValidationError`, not a stored bad value. The lookups run **together**, not one-after-another:

```
RunnableParallel(
    status=RunnableLambda(_field("status")),
    open_requests=RunnableLambda(_field("open_requests")),
    deadline=RunnableLambda(_field("deadline")),
)   # .invoke({"student": id}) -> {"status":..., "open_requests":..., ...}
```

Three 0.3s lookups take ≈0.3s together, not 0.9s. Prove it with `data/time_lookups.py`.

Hint 3 — the fallbacks, and how to tell one fired

`.with_fallbacks([...])` runs a backup runnable when the primary raises. Put both wrappers in `fallback.py`:

```
CANNED_REPLY = "Thanks for your message... a staff member will follow up..."

def give_canned_reply(_input):
    return CANNED_REPLY

def with_canned_reply(reply_chain):
    return reply_chain.with_fallbacks([RunnableLambda(give_canned_reply)])

def with_unsure_category(classifier_chain):
    # _unsure returns a TicketCategory with sure=False
    return classifier_chain.with_fallbacks([RunnableLambda(_unsure)])
```

(Write the lambdas as named functions if you're avoiding lambdas.) The trick for `used_fallback`: because `CANNED_REPLY` is a **fixed string**, after drafting you just check `reply.strip() == CANNED_REPLY`. No extra plumbing needed.

Hint 4 — streaming, and flagging for a human

Stream the reply so it appears live, and collect the full text for the record:

```
pieces = []
for piece in chain.stream({"category": c, "message": m, "context": ctx}):
    print(piece, end="", flush=True)   # flush=True is what makes it live
    pieces.append(piece)
reply = "".join(pieces)
```

`.stream()` works through the fallback too — if the model fails, the canned `RunnableLambda` streams its one string. Then flag for a human whenever the classifier wasn't sure **or** the message carries an urgent signal:

```
reasons = []
if not category.sure:
    reasons.append("classifier was not sure")
for word in ["today", "locked out", "dropped", "urgent", "immediately"]:
    if word in message.lower():
```

```
        reasons.append(f"urgent signal: {word}")
needs_human_review = len(reasons) > 0
```

Test this logic with no model — a clean message yields no flag, an urgent word or an unsure category yields one.

Still stuck? The whole pipeline, every file, is in the complete solution.